

Programação Orientada a Objetos em Java: Classes Abstratas e Interfaces

Dani Juliano Czelusniak, Prof. Dr.

Atualizado em 2026

1 Introdução à Abstração

Na Programação Orientada a Objetos (POO), a **Abstração** é o princípio pelo qual isolamos e focamos apenas nos aspectos essenciais de um domínio, ignorando os detalhes de implementação menos importantes ou complexos em um primeiro momento. Em vez de lidar com a complexidade direta de como algo é feito, a abstração nos permite definir *o que* deve ser feito.

Em Java, aplicamos o princípio da abstração em níveis arquiteturais utilizando dois recursos principais: **Classes Abstratas** e **Interfaces**. Ambos estabelecem **contratos** que outras classes (subclasses ou implementadoras) são obrigadas a seguir, garantindo padronização, organização e alto grau de reaproveitamento e segurança no código por meio do polimorfismo.

Neste material, detalharemos conceitualmente o que são classes abstratas, métodos abstratos e interfaces, demonstrando sua sintaxe, funcionamento dinâmico na memória e aplicabilidade em diagramas e códigos de cenários reais.

2 Classes Abstratas e Métodos Abstratos

2.1 O que é uma Classe Abstrata?

Uma **Classe Abstrata** é uma classe que não pode ser instanciada diretamente. Ou seja, você não pode usar o operador `new` para criar um objeto dela. Ela existe estritamente para ser herdada (estendida) por outras classes.

Utilizamos classes abstratas quando temos um conceito genérico (como “Veículo” ou “Animal”) que agrupa atributos e comportamentos comuns para suas subclasses, mas que é genérico demais para existir no mundo real por si só (não existe um objeto que seja *apenas* um Animal, ele será especificamente um Cachorro, Gato, etc.).

Para declarar uma classe como abstrata em Java, utilizamos a palavra-chave `abstract` antes de `class`.

2.2 O que é um Método Abstrato?

Um **Método Abstrato** é um método que é apenas *declarado* na classe pai, mas não possui corpo (implementação). Ele atua como uma obrigatoriedade: qualquer classe filha concreta (não-abstrata) que herdar da classe pai **deve** obrigatoriamente fornecer a implementação (*override*) para esse método.

Métodos abstratos só podem existir dentro de classes abstratas (ou interfaces). Eles são finalizados com ponto e vírgula (;) no lugar de chaves ({}).

2.3 Exemplo Prático 1: Sistema de Formas Geométricas

Imagine um sistema¹ de desenho onde precisamos calcular a área de diversas formas. A regra matemática para calcular a área muda dependendo da forma (Círculo, Retângulo, etc.). Logo, a superclasse *Forma* não tem como saber calcular a própria área, mas ela pode obrigar suas filhas a saberem.

2.3.1 Recorte de um Diagrama de Classes ilustrativo

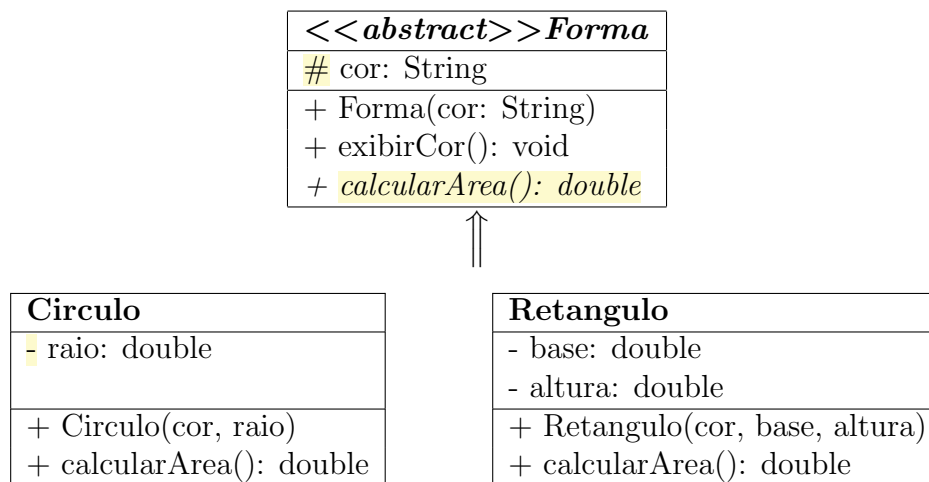


Figura 1: Fragmento de Diagrama de Classes com Classe e Método Abstrato (em itálico).

2.3.2 Implementação em Java

```

1 // 1. A Classe e marcada como abstract. Nao pode ser instanciada.
2 abstract class Forma {
3     protected String cor;
4
5     public Forma(String cor) {
6         this.cor = cor;
7     }
8
9     // Metodo concreto: herdado e utilizavel pelas filhas
10    normalmente
11    public void exibirCor() {
12        System.out.println("A cor desta forma e: " + this.cor);
13    }
14
15    // 2. Metodo abstrato: Nao tem corpo! Termina em ponto e
16    virgula.
17    // Obriga as subclasses a implementarem essa logica.
  
```

¹Esse é somente uma representação didática para um recorte de um diagrama de classes. A notação UML utilizada para tal não é seguida aqui em sua íntegra.

```

16     public abstract double calcularArea();
17 }
18
19 // 3. Subclasse concreta
20 class Circulo extends Forma {
21     private double raio;
22
23     public Circulo(String cor, double raio) {
24         super(cor);
25         this.raio = raio;
26     }
27
28     // 4. Implementacao obrigatoria do metodo abstrato
29     @Override
30     public double calcularArea() {
31         return Math.PI * (raio * raio);
32     }
33 }
34
35 // 5. Outra subclasse concreta
36 class Retangulo extends Forma {
37     private double base;
38     private double altura;
39
40     public Retangulo(String cor, double base, double altura) {
41         super(cor);
42         this.base = base;
43         this.altura = altura;
44     }
45
46     // Implementacao obrigatoria com regra matematica diferente
47     @Override
48     public double calcularArea() {
49         return base * altura;
50     }
51 }
52
53 public class MainAbstract {
54     public static void main(String[] args) {
55         // Forma f = new Forma("Azul"); // ERRO! Nao compila.
56
57         Forma c1 = new Circulo("Vermelho", 5.0);
58         Forma r1 = new Retangulo("Verde", 4.0, 5.0);
59
60         c1.exibirCor();
61         System.out.println("Area do Circulo: " + c1.calcularArea
62         ());
63
64         r1.exibirCor();
65         System.out.println("Area do Retangulo: " + r1.
66         calcularArea());

```

65
66

```
}  
}
```

Listing 1: Exemplo de Classes e Métodos Abstratos

3 Interfaces

3.1 O que é uma Interface?

Uma **Interface** em Java é uma referência de tipo, similar a uma classe, que atua como um contrato 100% abstrato (na sua forma mais clássica). Ela dita *o que* uma classe deve fazer, mas nunca *como* deve fazer.

Diferente da herança de classes (onde o Java permite apenas herança simples com **extends**), uma classe em Java pode implementar **múltiplas interfaces** utilizando a palavra-chave **implements**. Isso permite uma forma segura de “herança múltipla de tipos”.

3.2 Características das Interfaces

- **Métodos:** Por padrão, todos os métodos declarados em uma interface são implicitamente **public** e **abstract**. (A partir do Java 8, interfaces podem ter métodos **default** ou **static** com corpo, mas o uso central continua sendo o contrato abstrato).
- **Atributos:** Qualquer variável em uma interface é implicitamente **public static final** (ou seja, é uma constante). Interfaces não guardam “estado” de objetos.
- **Instanciação:** Assim como classes abstratas, interfaces não podem ser instanciadas (**new**).

3.3 Exemplo Prático 2: Sistema de Segurança

Temos diferentes tipos de usuários num sistema², mas apenas alguns (como Gerentes e Administradores de TI) podem realizar login. Um “Cliente” herda de Usuário, mas não faz login administrativo. Uma interface **Autenticavel** resolve isso criando uma “habilidade” específica apenas para quem assiná-la.

3.3.1 Fragmento de um Diagrama de Classes

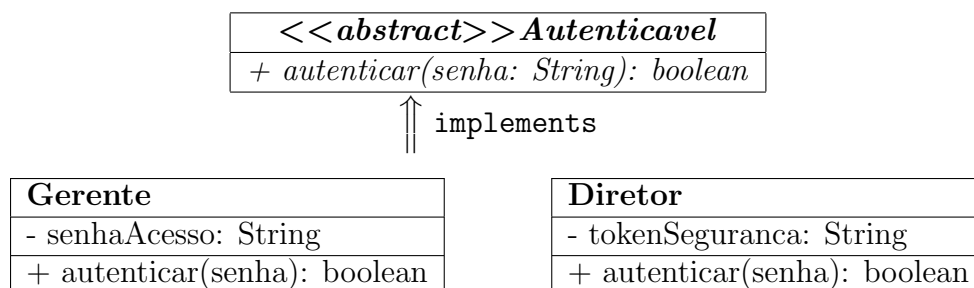


Figura 2: Fragmento de um Diagrama: Classes independentes implementando um contrato comum.

3.3.2 Implementação em Java

²Esse é somente uma representação didática para um recorte de um diagrama de classes. A notação UML utilizada para tal não é seguida aqui em sua íntegra.

```

1 // 1. Declaracao da interface usando a palavra reservada '
  interface'
2 interface Autenticavel {
3     // Metodo implicitamente public abstract
4     boolean autenticar(String senha);
5 }
6
7 // 2. Classe que assina o contrato da interface
8 class Gerente implements Autenticavel {
9     private String senhaAcesso;
10
11     public Gerente(String senhaAcesso) {
12         this.senhaAcesso = senhaAcesso;
13     }
14
15     // 3. OBRIGATORIO: fornecer a implementacao do metodo da
    interface
16     @Override
17     public boolean autenticar(String senha) {
18         // Regra simples: comparar strings
19         return this.senhaAcesso.equals(senha);
20     }
21 }
22
23 // 4. Outra classe com regra totalmente diferente para o mesmo
    contrato
24 class Diretor implements Autenticavel {
25     private String tokenSeguranca;
26
27     public Diretor(String token) {
28         this.tokenSeguranca = token;
29     }
30
31     @Override
32     public boolean autenticar(String senha) {
33         // Diretor precisa concatenar a senha com um token
    especial
34         String acessoRequerido = senha + "-admin";
35         return this.tokenSeguranca.equals(acessoRequerido);
36     }
37 }
38
39 public class MainInterface {
40     public static void main(String[] args) {
41         Gerente g = new Gerente("1234");
42         Diretor d = new Diretor("senhaForte-admin");
43
44         // Polimorfismo usando a interface como tipo de
    referencia!
45         Autenticavel login1 = g;
46         Autenticavel login2 = d;

```

```
47         System.out.println("Gerente logou? " + login1.autenticar(  
48         "1234"));  
49         System.out.println("Diretor logou? " + login2.autenticar(  
50         "senhaForte"));  
51     }
```

Listing 2: Uso de Interfaces para criar contratos de comportamento

4 Classes Abstratas vs Interfaces: Quando usar qual?

Embora pareçam ter o mesmo objetivo, elas são conceitualmente e arquiteturalmente diferentes:

Classes Abstratas	Interfaces
Uma classe pode estender apenas UMA classe abstrata (herança simples).	Uma classe pode implementar MÚLTIPLAS interfaces.
Podem ter métodos abstratos E métodos concretos já implementados.	(Clássico) Só possuem métodos abstratos. Define <i>apenas</i> o contrato.
Podem ter atributos de instância para guardar estado (nome, idade).	Só possuem constantes (<code>public static final</code>). Não guardam estado.
Usa-se para indicar IDENTIDADE (Ex: Cachorro <i>É UM</i> Animal).	Usa-se para indicar CAPACIDADE (Ex: Arquivo <i>É IMPRIMÍVEL</i>).

Tabela 1: Comparativo direto entre Classes Abstratas e Interfaces.

4.1 Adendo: Limitações de Herança em Java

4.1.1 Por que uma classe pode estender apenas UMA classe abstrata?

Em Java (e em várias outras linguagens orientadas a objetos, como C#), uma classe pode estender apenas **uma** única classe (seja ela concreta ou abstrata) devido a uma decisão arquitetural fundamental da linguagem: **evitar a ambiguidade e a complexidade estrutural**.

O principal motivo para essa restrição é conhecido como o **Problema do Diamante** (*Diamond Problem*), além de questões relacionadas ao gerenciamento de estado na memória:

- **O Problema do Diamante (Ambiguidade de Métodos):** Classes abstratas podem conter métodos concretos (com implementação). Se uma classe D herdasse de duas classes abstratas B e C (que por sua vez herdam de A), e ambas reescrevessem o mesmo método `imprimir()`, qual versão o compilador deveria executar ao chamar `D.imprimir()`? Para evitar essa confusão, o Java proíbe a herança múltipla de classes.
- **O Problema do Estado (Atributos e Construtores):** Classes abstratas podem declarar variáveis de instância (estado) e possuem construtores. Se uma classe pudesse herdar de duas classes abstratas, haveria conflitos na alocação de memória para atributos com o mesmo nome e na chamada ao `super()` no construtor.

4.1.2 Herança múltipla em Java

Em Java, a herança múltipla não é suportada para **classes**, mas é plenamente suportada para **interfaces**.

- **Herança de Classes (Não permitida):** Uma classe jamais pode usar a palavra-chave `extends` para herdar de mais de uma classe, blindando a linguagem contra conflitos de memória e o Problema do Diamante na inicialização de construtores.

- **Herança de Interfaces (Permitida):** Uma classe pode usar a palavra-chave `implements` para adotar múltiplos contratos simultaneamente (ex: `class Smartphone extends Dispositivo implements Camera, Telefone, Navegador`). Além disso, uma própria interface pode herdar de várias outras usando `extends`.

Nota sobre o Java 8+: Com a introdução dos métodos `default` nas interfaces, o Java passou a suportar a herança múltipla de *comportamento* (mas não de estado, pois interfaces continuam sem poder ter atributos de instância). Se houver conflito de código entre dois métodos `default` de interfaces diferentes, o compilador gera um erro e exige que o desenvolvedor sobrescreva o método conflitante na classe filha, obrigando-o a escolher ou reescrever a lógica para resolver a ambiguidade manualmente.

5 Lista de Exercícios Práticos

A fim de consolidar os conceitos de Classes Abstratas e Interfaces, resolva e analise os 6 exercícios a seguir, classificados por nível de dificuldade. Todos os códigos foram estruturados para execução em compiladores online (como o GDB Online), contendo apenas a classe Main como pública ao final do arquivo.

5.1 Nível Fácil

Exercício 1: Formas Geométricas (Classe e Método Abstrato)

Enunciado: Crie uma classe abstrata Forma com o método abstrato `calcularArea()`. Crie duas subclasses concretas, `Quadrado` e `Circulo`, que implementam a regra matemática de área. No método main, instancie as formas e exiba a área.

```
1 // 1. Classe Abstrata (Nao pode ser instanciada)
2 abstract class Forma {
3     // Metodo abstrato: sem corpo, obriga as filhas a
    implementarem
4     public abstract double calcularArea();
5 }
6
7 // 2. Subclasses Concretas
8 class Quadrado extends Forma {
9     private double lado;
10
11     public Quadrado(double lado) { this.lado = lado; }
12
13     @Override
14     public double calcularArea() {
15         return lado * lado;
16     }
17 }
18
19 class Circulo extends Forma {
20     private double raio;
21
22     public Circulo(double raio) { this.raio = raio; }
23
24     @Override
25     public double calcularArea() {
26         return Math.PI * raio * raio;
27     }
28 }
29
30 // 3. Classe Principal (Unica 'public' do arquivo)
31 public class Main {
32     public static void main(String[] args) {
33         Forma q = new Quadrado(5.0);
34         Forma c = new Circulo(3.0);
35
36         System.out.println("Area do Quadrado: " + q.calcularArea
37                             ());
```

```

37         System.out.println("Area do Circulo: " + c.calcularArea()
38     );
39 }

```

Listing 3: Resolução: Exercício 1 - Fácil

Exercício 2: Contrato de Impressão (Interface)

Enunciado: Crie uma interface `Imprimivel` com o método `imprimir()`. Crie as classes `Documento` e `Foto` que assinam este contrato e implementam o método. Teste no `main`.

```

1 // 1. Interface (Contrato)
2 interface Imprimivel {
3     void imprimir();
4 }
5
6 // 2. Classes implementando o contrato
7 class Documento implements Imprimivel {
8     @Override
9     public void imprimir() {
10         System.out.println("Imprimindo documento de texto em
11         preto e branco...");
12     }
13 }
14 class Foto implements Imprimivel {
15     @Override
16     public void imprimir() {
17         System.out.println("Imprimindo foto em papel fotografico
18         colorido...");
19     }
20 }
21 // 3. Classe Principal
22 public class Main {
23     public static void main(String[] args) {
24         Imprimivel doc = new Documento();
25         Imprimivel foto = new Foto();
26
27         doc.imprimir();
28         foto.imprimir();
29     }
30 }

```

Listing 4: Resolução: Exercício 2 - Fácil

5.2 Nível Médio

Exercício 3: Folha de Pagamento (Abstração com Construtores)

Enunciado: Classes abstratas podem ter estado e construtores. Crie `Funcionario` abstrata com `nome`, `salarioBase` e o método abstrato `calcularSalario()`. Crie `Gerente` (ganha bônus de 20%) e `Vendedor` (ganha comissão fixa de R\$ 500). Imprima os salários no `main`.

```
1 abstract class Funcionario {
2     protected String nome;
3     protected double salarioBase;
4
5     // Construtor na classe abstrata
6     public Funcionario(String nome, double salarioBase) {
7         this.nome = nome;
8         this.salarioBase = salarioBase;
9     }
10
11     public abstract double calcularSalario();
12
13     public void exibirInfo() {
14         System.out.println(nome + " - Salario: R$ " +
15             calcularSalario());
16     }
17 }
18
19 class Gerente extends Funcionario {
20     public Gerente(String nome, double salarioBase) { super(nome,
21         salarioBase); }
22
23     @Override
24     public double calcularSalario() {
25         return salarioBase + (salarioBase * 0.20); // 20% bonus
26     }
27 }
28
29 class Vendedor extends Funcionario {
30     public Vendedor(String nome, double salarioBase) { super(nome
31         , salarioBase); }
32
33     @Override
34     public double calcularSalario() {
35         return salarioBase + 500.0; // Comissao fixa
36     }
37 }
38
39 public class Main {
40     public static void main(String[] args) {
41         Funcionario g = new Gerente("Alice", 5000.0);
42         Funcionario v = new Vendedor("Bruno", 2000.0);
43
44         g.exibirInfo();
```

```

42         v.exibirInfo();
43     }
44 }

```

Listing 5: Resolução: Exercício 3 - Médio

Exercício 4: Herança Múltipla de Comportamento (Múltiplas Interfaces)

Enunciado: O Java permite implementar várias interfaces. Crie *Voador* (método *voar*) e *Nadador* (método *nadar*). Crie a classe *Pato* que implementa ambas. Crie *Pinguim* que implementa apenas *Nadador*.

```

1 interface Voador { void voar(); }
2 interface Nadador { void nadar(); }
3
4 // Implementa duas interfaces!
5 class Pato implements Voador, Nadador {
6     @Override
7     public void voar() { System.out.println("Pato voando baixo..."); }
8
9     @Override
10    public void nadar() { System.out.println("Pato nadando no lago..."); }
11 }
12
13 class Pinguim implements Nadador {
14     @Override
15     public void nadar() { System.out.println("Pinguim nadando super rapido nas geleiras..."); }
16 }
17
18 public class Main {
19     public static void main(String[] args) {
20         Pato donald = new Pato();
21         Pinguim capitao = new Pinguim();
22
23         donald.voar();
24         donald.nadar();
25
26         // capitao.voar(); // Erro! Pinguim nao assinou o
contrato Voador
27         capitao.nadar();
28     }
29 }

```

Listing 6: Resolução: Exercício 4 - Médio

5.3 Nível Difícil

Exercício 5: Banco Genérico (Polimorfismo com Vetores e Abstração)

Enunciado: Crie `ContaBancaria` (abstrata) com saldo, método concreto `depositar()` e abstrato `sacar()`. `ContaCorrente` taxa R\$ 2 por saque. `ContaPoupanca` não taxa, mas não permite saldo negativo. No main, crie um vetor de `ContaBancaria`, insira os dois tipos, deposite R\$ 100 em todas e saque R\$ 50 usando um laço for.

```
1 abstract class ContaBancaria {
2     protected double saldo;
3
4     // Metodo concreto dentro de classe abstrata
5     public void depositar(double valor) {
6         this.saldo += valor;
7         System.out.println("Deposito de R$" + valor + " | Saldo:
R$" + saldo);
8     }
9
10    public abstract void sacar(double valor);
11 }
12
13 class ContaCorrente extends ContaBancaria {
14     @Override
15     public void sacar(double valor) {
16         this.saldo -= (valor + 2.0); // Taxa de 2 reais
17         System.out.println("Saque (CC). Saldo: R$" + saldo);
18     }
19 }
20
21 class ContaPoupanca extends ContaBancaria {
22     @Override
23     public void sacar(double valor) {
24         if (this.saldo >= valor) {
25             this.saldo -= valor;
26             System.out.println("Saque (Poupanca). Saldo: R$" +
saldo);
27         } else {
28             System.out.println("Saldo insuficiente na Poupanca!");
29         }
30     }
31 }
32
33 public class Main {
34     public static void main(String[] args) {
35         ContaBancaria[] contas = new ContaBancaria[2];
36         contas[0] = new ContaCorrente();
37         contas[1] = new ContaPoupanca();
38
39         for(ContaBancaria c : contas) {
40             c.depositar(100.0);
41             c.sacar(50.0);
```

```
42         System.out.println("--");
43     }
44 }
45 }
```

Listing 7: Resolução: Exercício 5 - Difícil

Exercício 6: Arquitetura Híbrida (Extends + Implements)

Enunciado: Crie a interface Autenticavel com o método boolean login(String senha). Crie a classe abstrata Usuario com atributo nome. Crie a classe Cliente (herda de Usuario) e a classe Diretor (herda de Usuario e implementa Autenticavel, senha="123"). No main, simule um sistema que aceita apenas quem é Autenticavel.

```
1 interface Autenticavel {
2     boolean login(String senha);
3 }
4
5 abstract class Usuario {
6     protected String nome;
7     public Usuario(String nome) { this.nome = nome; }
8 }
9
10 // Cliente e apenas um Usuario, nao entra no sistema restrito
11 class Cliente extends Usuario {
12     public Cliente(String nome) { super(nome); }
13 }
14
15 // Diretor e um Usuario E TAMBEM assina o contrato Autenticavel
16 class Diretor extends Usuario implements Autenticavel {
17     public Diretor(String nome) { super(nome); }
18
19     @Override
20     public boolean login(String senha) {
21         System.out.println("Diretor " + nome + " tentando acesso
22         ...");
23         return senha.equals("12345");
24     }
25 }
26
27 class SistemaSeguranca {
28     // Metodo que so aceita objetos que assinaram a interface
29     // Autenticavel
30     public void autenticar(Autenticavel a, String senhaDigitada)
31     {
32         if(a.login(senhaDigitada)) {
33             System.out.println("Acesso Liberado pelo Sistema!");
34         } else {
35             System.out.println("Acesso Negado!");
36         }
37     }
38 }
39
40 public class Main {
41     public static void main(String[] args) {
42         SistemaSeguranca sistema = new SistemaSeguranca();
43         Diretor diretor = new Diretor("Carlos");
44         Cliente cliente = new Cliente("Joao");
45
46         // O polimorfismo de interface age aqui:
```

```
44     sistema.autenticar(diretor, "12345");
45     sistema.autenticar(diretor, "errada");
46
47     // sistema.autenticar(cliente, "000"); // ERRO DE
48     COMPILACAO!
49     // Cliente nao e Autenticavel. O compilador protege o
50     sistema.
    }
```

Listing 8: Resolução: Exercício 6 - Difícil